

Finite-State Morphological Parsing

29.01.2026

CONTENT

- Finite-State Morphological Parsing
- The Lexicon and Morphotactics
- Morphological Parsing with Finite-State Transducers
- Combining FST Lexicon and Rules
- Lexicon-free FSTs: The Porter Stemmer

Finite-State Morphological Parsing

- Parsing English morphology

Input	Morphological parsed output
cats	cat +N +PL
cat	cat +N +SG
cities	city +N +PL
geese	goose +N +PL
goose	(goose +N +SG) or (goose +V)
gooses	goose +V +3SG
merging	merge +V +PRES-PART
caught	(caught +V +PAST-PART) or (catch +V +PAST)

Contd...

- We need at least the following to build a morphological parser:
 1. **Lexicon**: the list of stems and affixes, together with basic information about them (Noun stem or Verb stem, etc.)
 2. **Morphotactics**: the model of morpheme ordering that explains which classes of morphemes can follow other classes of morphemes inside a word. E.g., the rule that English plural morpheme follows the noun rather than preceding it.
 3. **Orthographic rules**: these **spelling rules** are used to model the changes that occur in a word, usually when two morphemes combine (e.g., the $y \rightarrow ie$ spelling rule changes *city* + *-s* to *cities*).

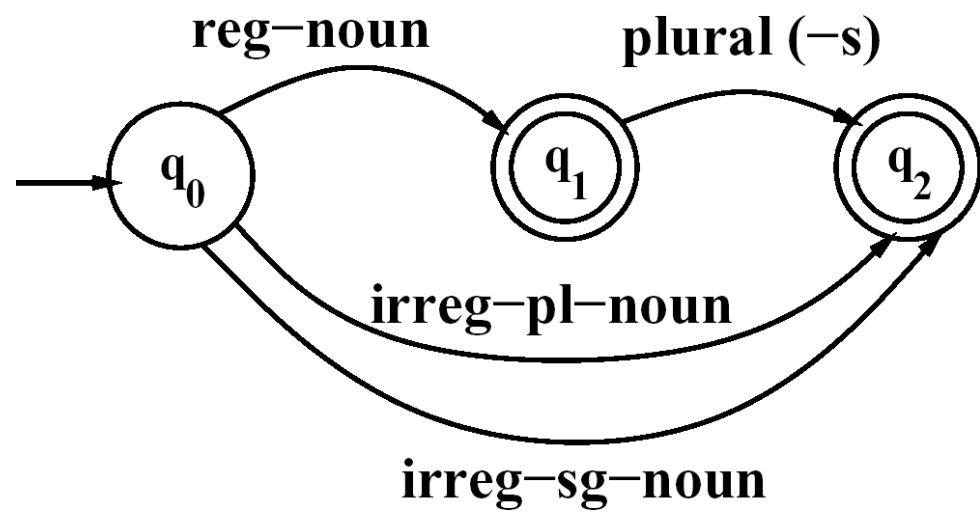
The Lexicon and Morphotactics

- A lexicon is a repository for words.
 - The simplest one would consist of an explicit list of every word of the language.

Inconvenient or impossible!

- Computational lexicons are usually structured with
 - a list of each of the stems and
 - Affixes of the language together with a representation of morphotactics telling us how they can fit together.
- The most common way of modeling morphotactics is the finite-state automaton.

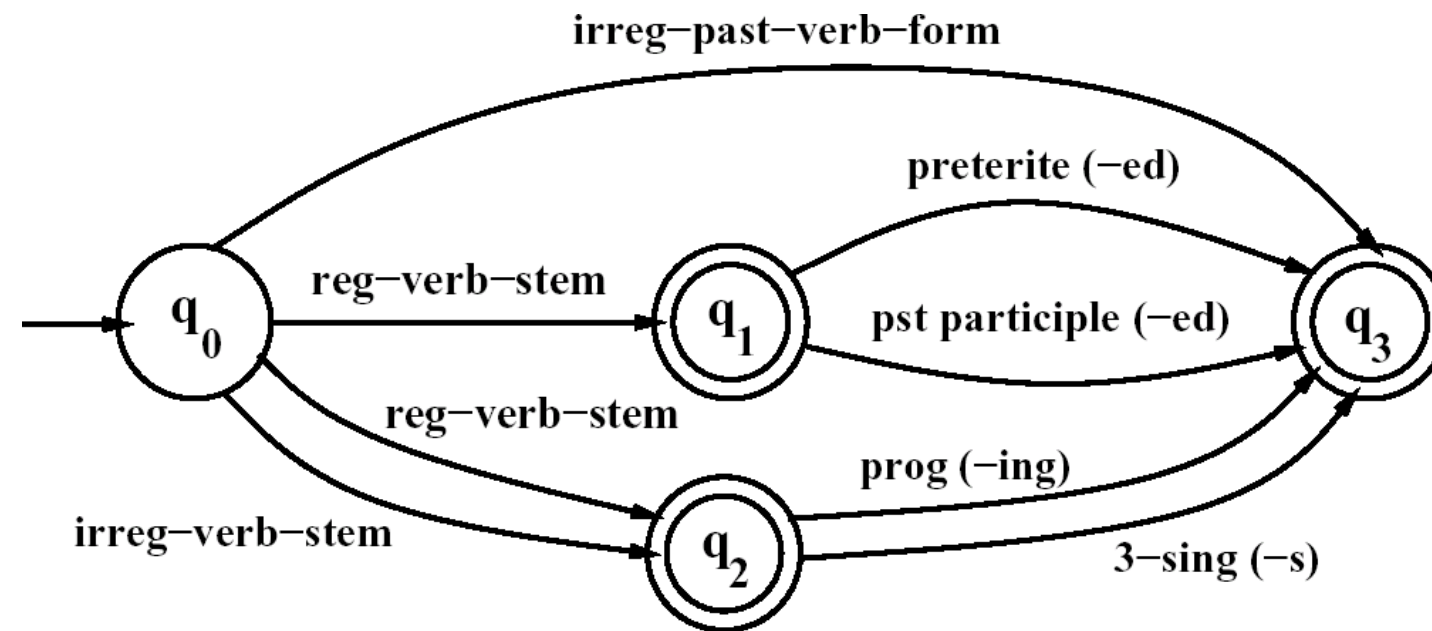
Contd...



Reg-noun	Irreg-pl-noun	Irreg-sg-noun	plural
fox fat fog fardvark	geese sheep Mice	goose sheep mouse	-s

An FSA for English nominal inflection

Contd...



An FSA for English verbal inflection

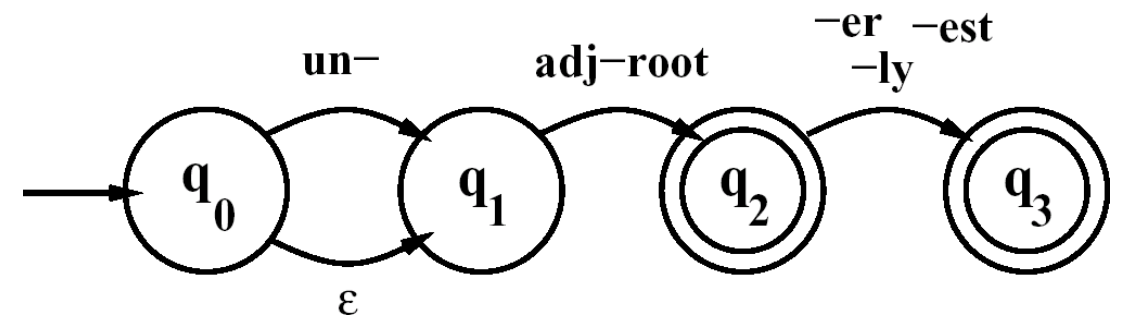
Contd...

Reg-verb-stem	Irreg-verb-stem	Irreg-past-verb	past	Past-part	Pres-part	3sg
walk fry talk impeach	Cut speak sing sang spoken	caught ate eaten	-ed	-ed	-ing	-s

Contd...

- English derivational morphology is more complex than English inflectional morphology, and so automata of modeling English derivation tends to be quite complex.
 - Some even based on CFG
- A small part of morphosyntactics of English adjectives

An FSA for a fragment of English adjective Morphology #1



Contd...

big, bigger, biggest

cool, cooler, coolest, coolly

red, redder, reddest

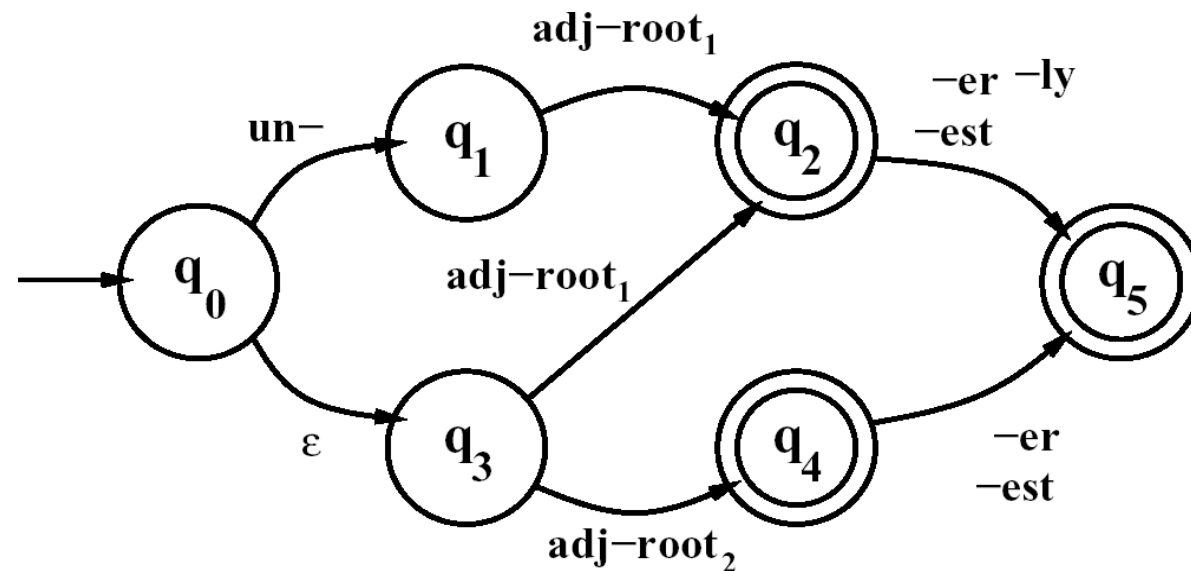
clear, clearer, clearest, clearly, unclear, unclearly

happy, happier, happiest, happily, unhappy, unhappier, unhappiest,
unhappily

real, unreal, really

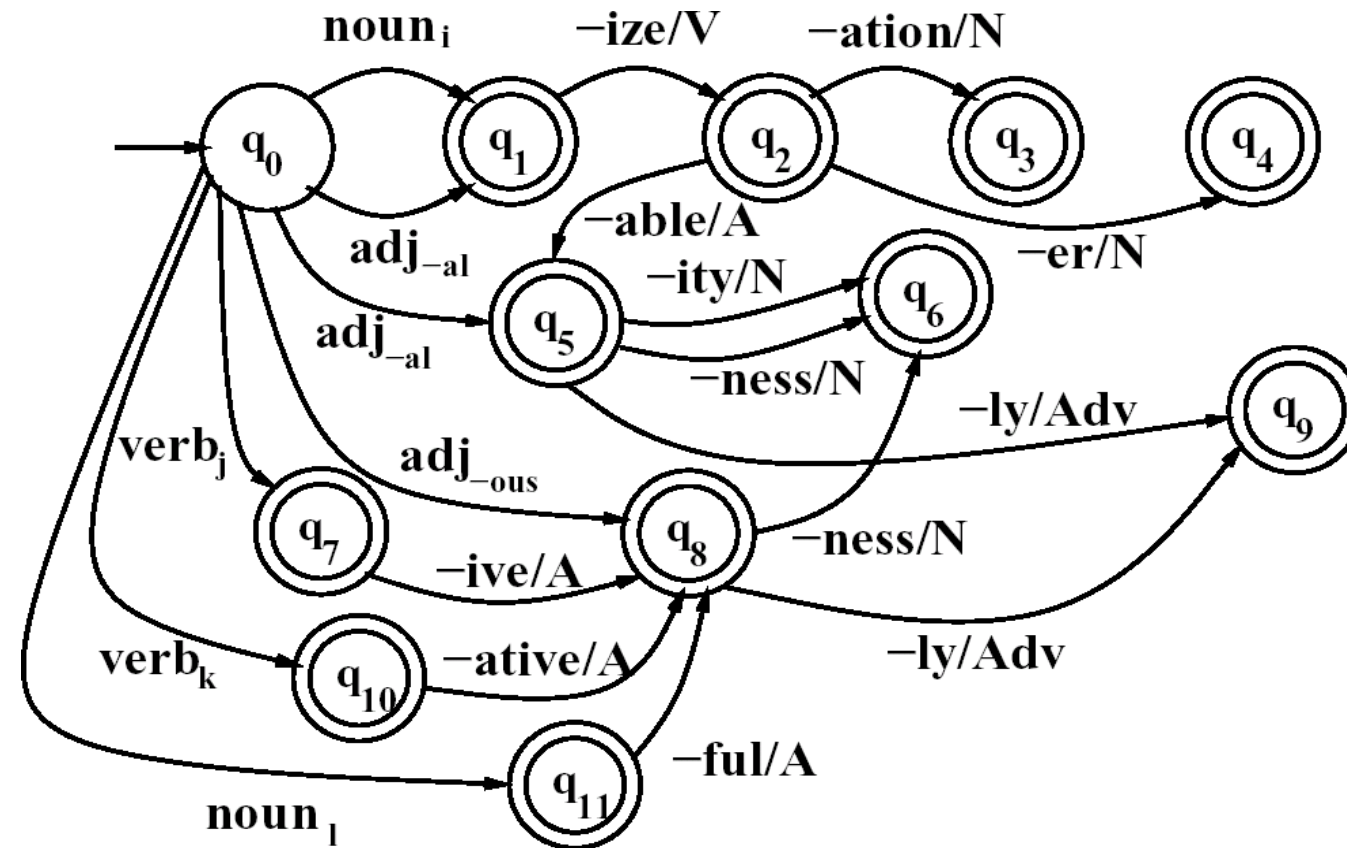
Contd...

- The FSA#1 recognizes all the listed adjectives, and ungrammatical forms like *unbig*, *redly*, and *realest*.
- Thus #1 is revised to become #2.
- The complexity is expected from English derivation.



An FSA for a fragment of English adjective Morphology #2

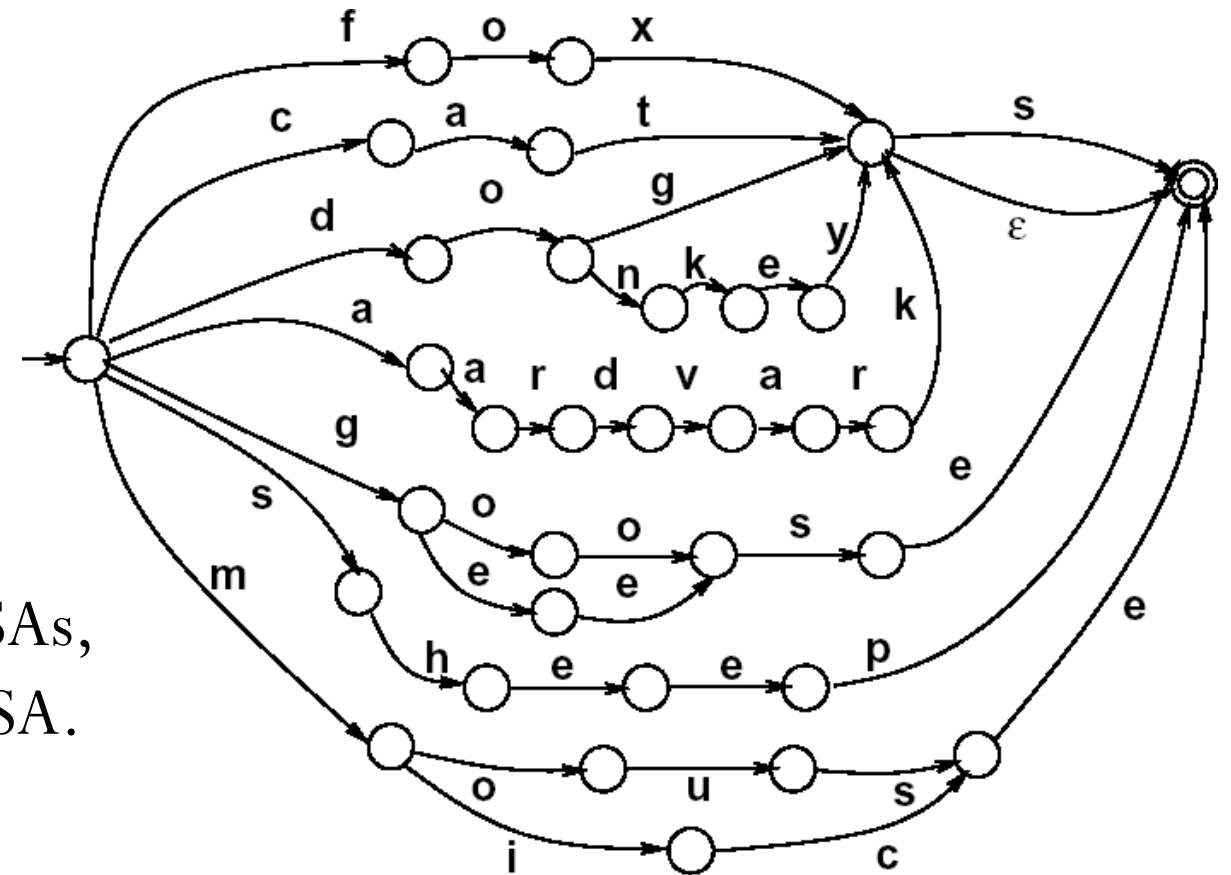
Contd...



An FSA for another fragment of English derivational morphology

Contd...

- We can now use these FSAs to solve the problem of **morphological recognition**:
 - Determining whether an input string of letters makes up a legitimate English word or not
 - We do this by taking the morphotactic FSAs, and plugging in each “sub-lexicon” into the FSA.
 - The resulting FSA can then be defined as the level of the individual letter.



ACTIVE LEARNING ACTIVITY

ONE MINUTE PAPER

Find Morphological parsed output for the following words

1. Sheep
2. Goats
3. Walking
4. Mice

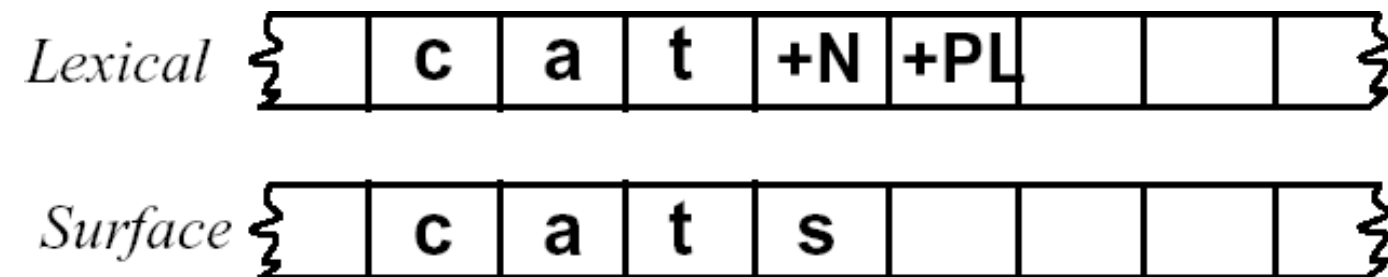
<https://www.menti.com/alp7j5bnrfgb>

Morphological Parsing with Finite-State Transducers

- Given the input, for example, *cats*, we would like to produce cat +N +PL.
- Two-level morphology
 - Representing a word as a correspondence between a **lexical level**
 - Representing a simple concatenation of morphemes making up a word, and
 - The **surface level**
 - Representing the actual spelling of the final word.

Contd...

- Morphological parsing is implemented by building mapping rules that maps letter sequences like *cats* on the surface level into morpheme and features sequence like *cat +N +PL* on the lexical level.
- The automaton we use for performing the mapping between these two levels is the finite-state transducer or FST.
 - A transducer maps between one set of symbols and another;
 - An FST does this via a finite automaton.



Contd...

- Thus an FST can be seen as a two-tape automaton which **recognizes** or **generates** *pairs* of strings.
- The FST has a more general function than an FSA:
 - An FSA defines a formal language
 - An FST defines a relation between sets of strings.
- Another view of an FST:
 - A machine reads one string and generates another.

four-fold way of thinking about transducers

- **FST as recognizer:**
 - a transducer that takes a pair of strings as input and output *accept* if the string-pair is in the string-pair language, and a *reject* if it is not.
- **FST as generator:**
 - a machine that outputs pairs of strings of the language. Thus the output is a yes or no, and a pair of output strings.
- **FST as transducer:**
 - A machine that reads a string and outputs another string.
- **FST as set relater:**
 - A machine that computes relation between sets.

Contd...

- A formal definition of FST (based on the **Mealy machine** extension to a simple FSA):
 - Q : a finite set of N states $q_0, q_1, \dots, q_N$
 - Σ : a finite alphabet of complex symbols. Each complex symbol is composed of an input-output pair $i : o$; one symbol I from an input alphabet I , and one symbol o from an output alphabet O , thus $\Sigma \subseteq I \times O$.
 I and O may each also include the epsilon symbol ϵ .
 - q_0 : the start state
 - F : the set of final states, $F \subseteq Q$
 - $\delta(q, i:o)$: the transition function or transition matrix between states. Given a state $q \in Q$ and complex symbol $i:o \in \Sigma$, $\delta(q, i:o)$ returns a new state $q' \in Q$. δ is thus a relation from $Q \times \Sigma$ to Q .

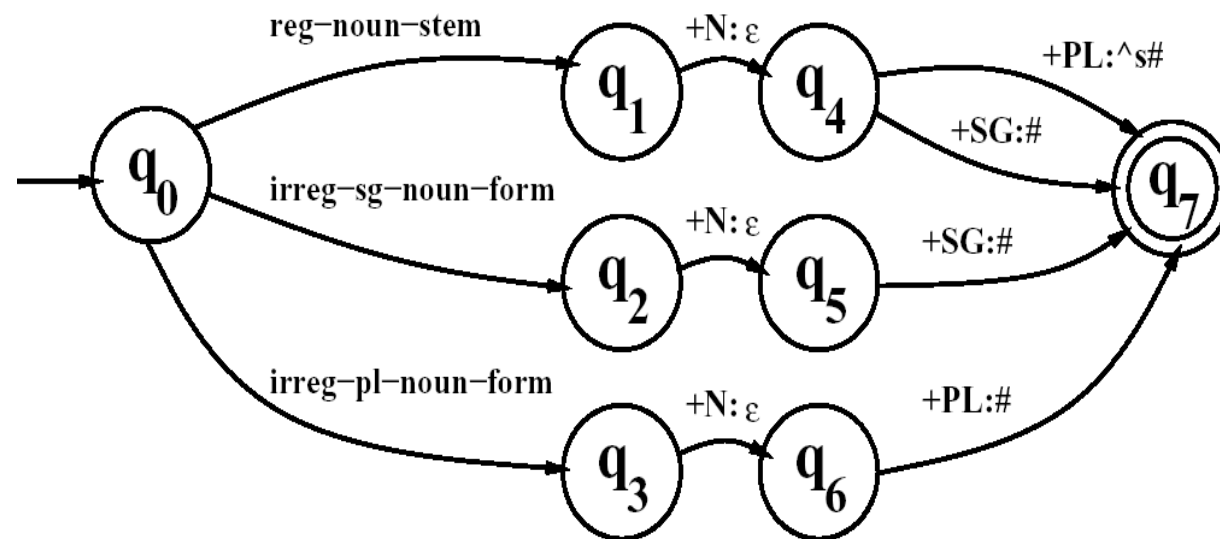
Contd...

- FSAs are isomorphic to regular languages, FSTs are isomorphic to **regular relations**.
- Regular relations are sets of pairs of strings, a natural extension of the regular language, which are sets of strings.
- FSTs are closed under union, but generally they are not closed under difference, complementation, and intersection.
- Two useful closure properties of FSTs:
 - **Inversion:** If T maps from I to O , then the inverse of T , T^{-1} maps from O to I .
 - **Composition:** If T_1 is a transducer from I_1 to O_1 and T_2 a transducer from I_2 to O_2 , then $T_1 \circ T_2$ maps from I_1 to O_2

Contd...

- Inversion is useful because it makes it easy to convert a FST-as-parser into an FST-as-generator.
- Composition is useful because it allows us to take two transducers than run in series and replace them with one complex transducer.

$$- T_1 \circ T_2(S) = T_2(T_1(S))$$



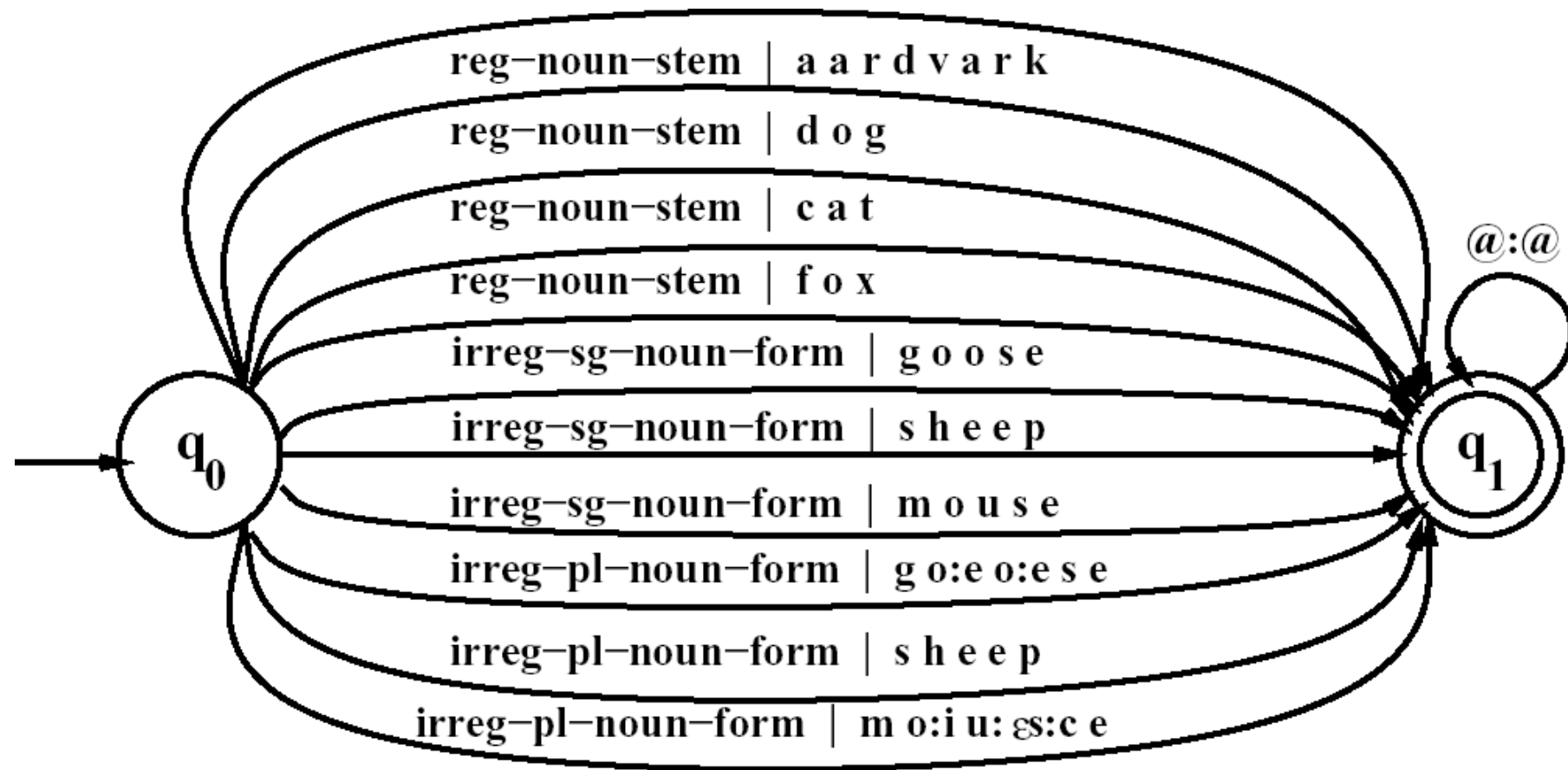
A transducer for English nominal number inflection T_{num}

Contd...

Reg-noun	Irreg-pl-noun	Irreg-sg-noun
fox fat	g o:e o:e s e	goose
fog	sheep	sheep
aardvark	m o:i u:ɛs:c e	mouse

Contd...

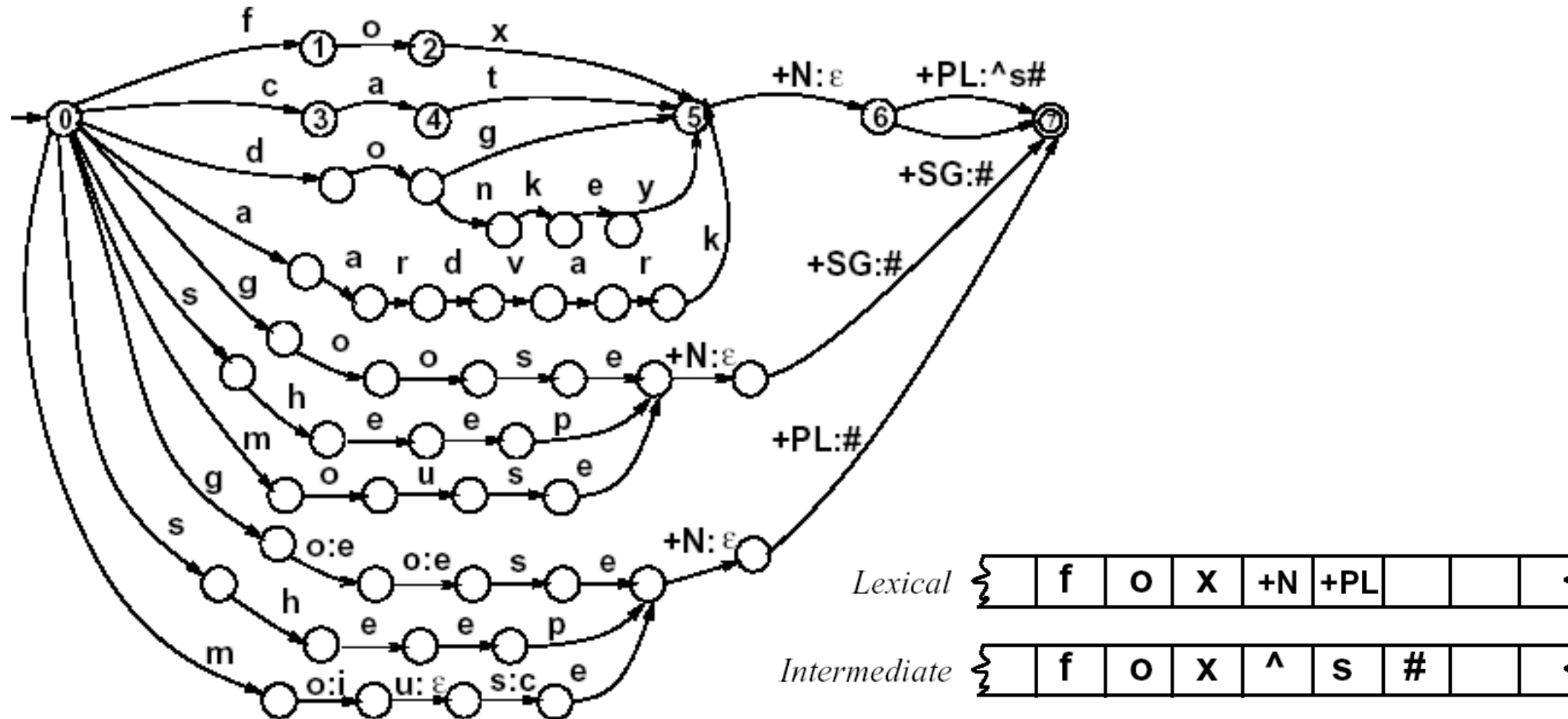
The transducer T_{stems} , which maps roots to their root-class



Contd...

A fleshed-out English nominal inflection FST

$$T_{lex} = T_{num} \circ T_{stems}$$



Orthographic Rules and FSTs

- **Spelling rules (or orthographic rules)**

Name	Description of Rule	Example
Consonant doubling	1-letter consonant doubled before <i>-ing</i> / <i>-ed</i>	beg/begging
E deletion	Silent e dropped before <i>-ing</i> and <i>-ed</i>	make/making
E insertion	e added after <i>-s</i> , <i>-z</i> , <i>-x</i> , <i>-ch</i> , <i>-sh</i> , before <i>-s</i>	watch/watches
Y replacement	<i>-y</i> changes to <i>-ie</i> before <i>-s</i> , <i>-i</i> before <i>-ed</i>	try/tries
K insertion	Verb ending with <i>vowel</i> + <i>-c</i> add <i>-k</i>	panic/panicked

Contd...

- These spelling changes can be thought as taking as input a simple concatenation of morphemes and producing as output a slightly-modified concatenation of morphemes.

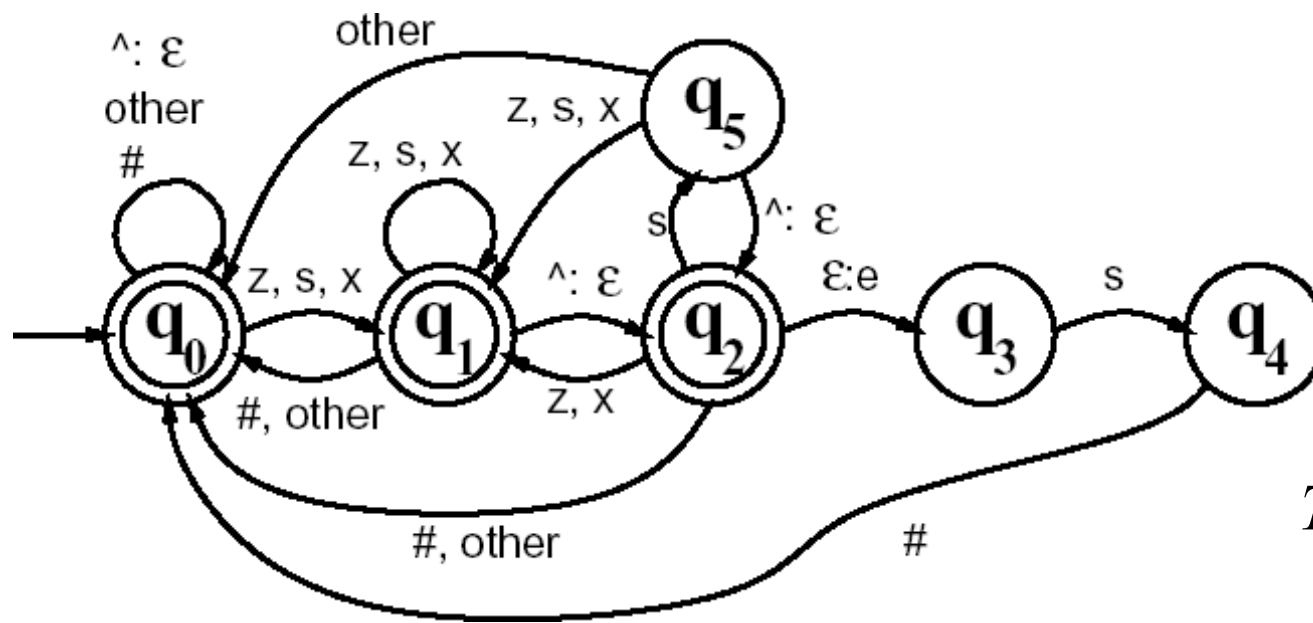
<i>Lexical</i>		f	o	x	+N	+PL			
<i>Intermediate</i>		f	o	x	^	s	#		
<i>Surface</i>		f	o	x	e	s			

-
- “insert an e on the surface tape just when the lexical tape has a morpheme ending in x (or z , etc) and the next morphemes is $-s$ ”

$$\varepsilon \rightarrow e / \left\{ \begin{array}{c} x \\ s \\ z \end{array} \right\} _ s\#$$

- “rewrite a to b when it occurs between c and d ”

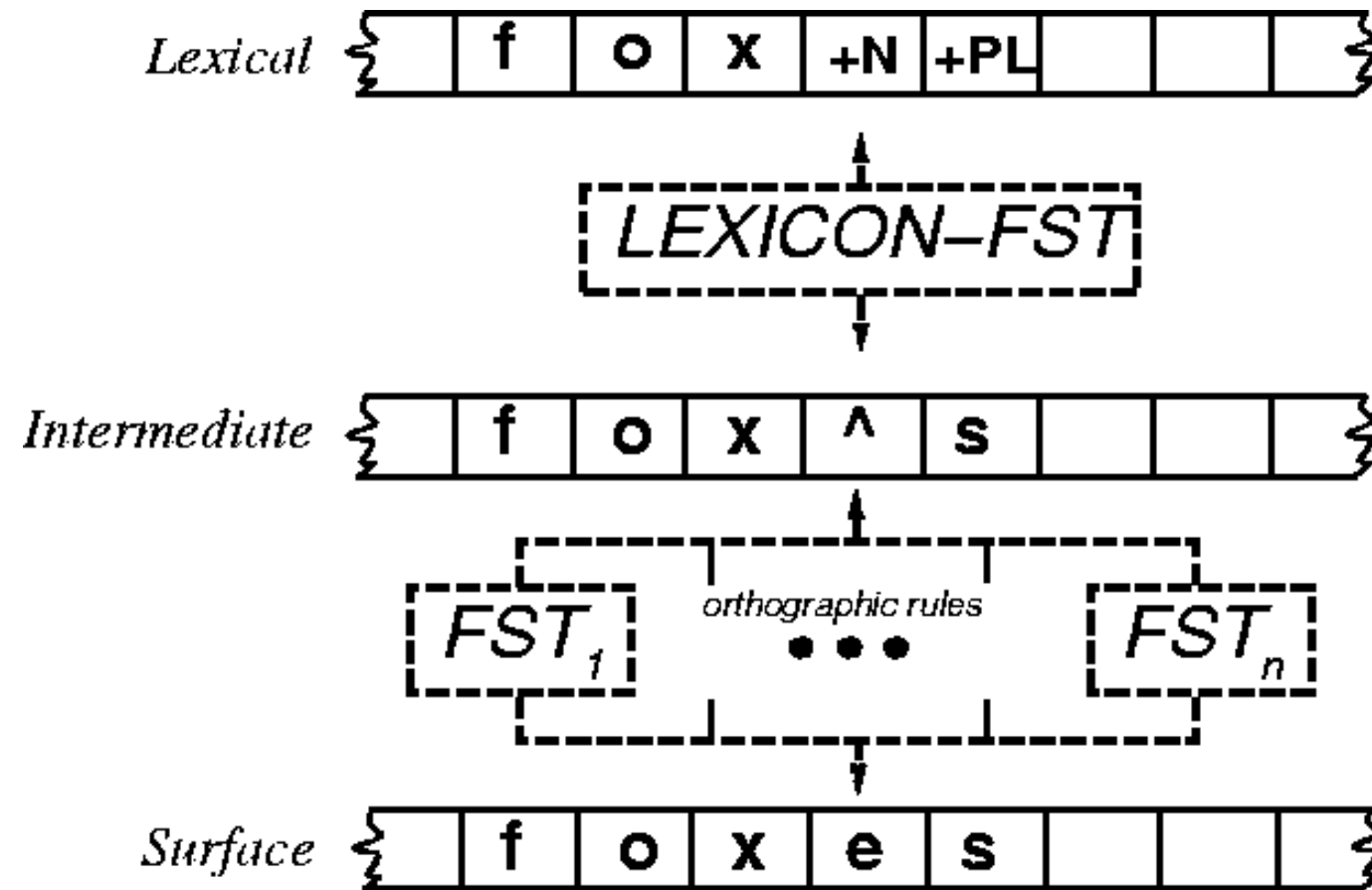
$$a \rightarrow b / c _ d$$

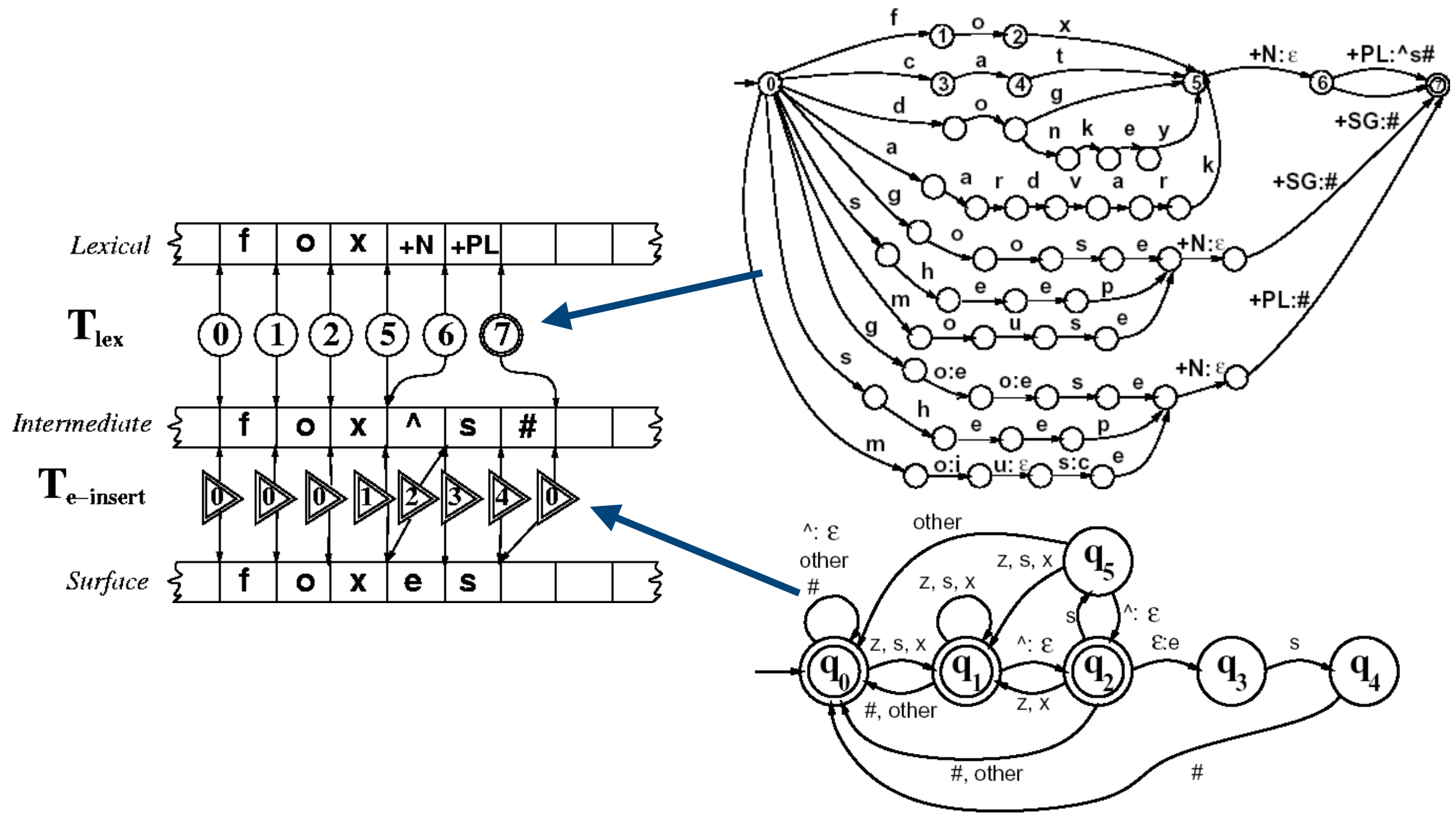


The transducer for the E-insertion rule

State \ Input	s : s	x : x	z : z	^ : ε	ε : e	#	other
q_0 :	1	1	1	0	-	0	0
q_1 :	1	1	1	2	-	0	0
q_2 :	5	1	1	0	3	0	0
q_3	4	-	-	-	-	-	-
q_4	-	-	-	-	-	0	-
q_5	1	1	1	2	-	-	0

Combining FST Lexicon and Rules





Contd...

- The power of FSTs is that the exact same cascade with the same state sequences is used
 - when machine is generating the surface form from the lexical tape, or
 - When it is parsing the lexical tape from the surface tape.
- Parsing can be slightly more complicated than generation, because of the problem of **ambiguity**.
 - For example, *foxes* could be fox +V +3SGas well as fox +N +PL

Lexicon-Free FSTs: the Porter Stemmer

- Information retrieval
- One of the mostly widely used **stemming** algorithms is the simple and efficient Porter (1980) algorithm, which is based on a series of simple cascaded rewrite rules.
 - ATIONAL → ATE (e.g., relational → relate)
 - ING → ϵ if stem contains vowel (e.g., motoring → motor)
- Problem:
 - Not perfect: error of commission, omission
- Experiments have been made
 - Some improvement with smaller documents
 - Any improvement is quite small

Contd...

Errors of Commission

organization	organ
doing	doe
generalization	generic
numerical	numerous
policy	police
university	universe
negligible	negligent

Errors of Omission

European	Europe
analysis	analyzes
matrices	matrix
noise	noisy
sparse	sparsity
explain	explanation
urgency	urgent

Thank
you!